

Model Context Protocol (MCP) for Ansible Automation Platform

Version 1, 2026-09-18

Welcome

Introduction

Course Title: Course title FIXME

Description: Course description FIXME

Duration: ...

Objectives

On completing this course, you should be able to:

- Observable, measurable objective...
- Observable, measurable objective...
- Observable, measurable objective...

Prerequisites

This course assumes that you have the following prior experience:

- Course or experience...
- Course or experience...
- Course or experience...



Use of Artificial Intelligence: This publication contains content or media that is generated or assisted by AI tools. All the Training deliverables have primarily human authorship, and all content was reviewed by a human domain expert.

Lab Environment

FIXME: Instructions for accessing the lab environment for this hands-on based training.

Chapter 1: Understanding MCP & AAP Integration

Welcome to the learning section of this course. In this chapter, you will understand what the Model Context Protocol is, how it works, and how Ansible Automation Platform uses it to connect with AI tools.

We'll break this down into bite-sized sections that follow a simple pattern: **What is it?** → **Why is it needed?** → **How does it work?**

What You'll Learn

By the end of this chapter, you will be able to:

- Define the Model Context Protocol and understand its role in the AI ecosystem
- Understand the architecture of MCP and how different components interact
- Explain Ansible Automation Platform's MCP server and its capabilities
- Describe the complete communication workflow from user to AI to AAP
- Identify and describe the 6 AAP MCP toolsets
- Understand the end-to-end workflow for deploying and using MCP

1: Launching Dev Spaces from OpenShift

1. Launch the [OpenShift Web Console](#)
2. Select the **htpasswd_provider** button and use the credentials provided in the **Environment Details,window=_blank** page to login to the OpenShift console
3. Use the 9-block icon in the upper right to launch *Red Hat OpenShift Dev Spaces* directly from the OpenShift console header.

[\[OpenShift console header showing the 9-block application launcher icon for quick access to Dev Spaces\]](#) | [11-platform-dev-spaces/intro-dev_spaces_shortcut.png](#)



Alternatively, the direct URL is https://devspaces.{openshift_cluster_ingress_domain} also available in **Environment Details,window=_blank**.

Dev Spaces Workspace

Launching a specific Dev Spaces Workspace

Once you have launched the OpenShift Dev Spaces console, use the following steps to access your Ansible Dev Space workspace.

1. Login via OpenShift authentication using the same credentials as the OpenShift console

2. Select **Allow selected permissions** to authorize access and launch the console
3. Scroll down and select the workspace sample titled **Ansible Dev Space**

[Dev Spaces workspace selection page showing available workspaces including Ansible Dev Space] | 11-platform-dev-spaces/intro3.png



It will take a few minutes for the Dev Spaces instance to initialize. When it has loaded, a Visual Studio Code interface will appear.

[Dev Spaces workspace loading screen showing VS Code interface initialization] | 11-platform-dev-spaces/intro4.png

4. Click the **Trust Publishers and Install** button to trust the extension publishers that are included within the Dev Spaces workspace
5. Click the **Yes, I trust the Authors** button to trust the workspace authors (your instructors).
6. Wait a few minutes until the additional extension icons appear for Ansible and OpenShift on the left side.

[VS Code interface in Dev Spaces showing Ansible and OpenShift extension icons in the left sidebar] | 11-platform-dev-spaces/intro5.png

Ansible and OpenShift Extensions

For this lab, we have provided a [.code-workspace](#) configuration that included these extensions along with additional configurations to curate your development environment. See [the source of this workspace configuration](#).

1. Click on the **Ansible** extension icon to access the Ansible Development Tools extension (covered in a subsequent lab)

[Ansible extension interface in VS Code showing available Ansible development tools] | 11-platform-dev-spaces/intro6.png

Accessing a Terminal

The majority of the exercises in this lab will be performed using the Visual Studio Code terminal.

1. Open a new terminal by selecting the VS Code menu starting with the hamburger (3 horizontal lines) in the top left, then selecting **Terminal** → **New Terminal**

[VS Code terminal interface showing basic command line operations in the Dev Spaces environment] | 11-platform-dev-spaces/intro7.png

2. Explore the environment:

```
whoami
```

```
user
```

3. Check the Dev Spaces Workspace container OS release:

```
cat /etc/redhat-release
```

```
Red Hat Enterprise Linux release 9.7 (Plow)
```

4. This workspace provides access to **ansible** and additional developer tools:

```
ansible --version
```

```
ansible [core 2.19.3]
  config file = None
  configured module search path = ['/home/user/.ansible/plugins/modules',
'/usr/share/ansible/plugins/modules']
  ansible python module location = /usr/local/lib/python3.11/site-packages/ansible
  ansible collection location =
/home/user/.ansible/collections:/usr/share/ansible/collections
  executable location = /usr/local/bin/ansible
  python version = 3.11.13 (main, Aug 21 2025, 00:00:00) [GCC 11.5.0 20240719 (Red
Hat 11.5.0-11)] (/usr/bin/python3.11)
  jinja version = 3.1.6
  pyyaml version = 6.0.3 (with libyaml v0.2.5)
```

Ready? Start with Section 1 below!

What is MCP?

What is the Model Context Protocol?

The **Model Context Protocol (MCP)** is an open-source standard that provides a universal way for AI models to interact with external data and tools.

Think of it as **"USB-C for the AI ecosystem"** — just like USB-C works with many different devices without custom adapters, MCP works with many different AI models and tools without requiring custom integrations.

[MCP as USB-C for AI] | *mcp-concept.png*

Instead of building a unique integration every time you want an AI model to interact with a new system, MCP provides a standardized method. This means:

- AI models can connect to different data sources without model-specific code

- □ You can swap AI providers without rewriting integrations
- □ Security and permissions are handled consistently

Why is MCP Needed?

Imagine you want to use ChatGPT to query your servers, while a colleague uses Claude to do the same thing. Without a standard protocol:

- ChatGPT integration team writes custom code to connect ChatGPT to your servers
- Claude integration team writes completely different code to do the same thing
- You have duplicated code, duplicated bugs, and duplicated maintenance

With MCP, both ChatGPT and Claude connect to the same MCP server. No duplication. No redundancy.

The Real Problem MCP Solves

AI models like Claude or ChatGPT are trained on data **frozen at a point in time**. They don't know:

- What jobs ran in your Ansible Automation Platform **yesterday**
- Which servers are currently online in your inventory
- What security events happened **today**
- Real-time metrics or current status

MCP solves this by giving AI models **real-time access** to your live systems. They can ask questions and get current answers, not guesses based on training data.

How Does It Work? (Simple Version)

MCP works in three simple steps:

MCP Flow

1. **You ask** → You send a natural language request to your AI tool
2. **AI translates** → The AI understands your intent and calls the appropriate MCP tool
3. **MCP delivers** → The MCP server fetches real data and returns it to the AI

Example

```
You: "List my recent Ansible jobs"
□
AI: "I need to call the job_list tool with recent=true"
□
MCP Server: Queries AAP and returns the list of jobs
□
AI: "Here are your last 5 jobs: ..."
```

Next Steps

You understand **what** MCP is. Now let's understand **how** it's built — the architecture and components.

→ [Continue to Section 2: MCP Architecture](#)

MCP Architecture

What are the Building Blocks?

MCP is built from **four main components**. Let's break them down:

Component 1: The AI Client (The Requester)

This is where **you** interact with the AI. It could be:

- **Claude Desktop** — A desktop application by Anthropic
- **Cursor IDE** — A code editor with AI built in
- **LibreChat** — A web-based chat interface
- **ChatGPT** — OpenAI's chatbot
- **Any MCP-compatible tool** you're using

Role: You enter natural language requests here. "Show me failed jobs from yesterday." The client sends your request to the AI model.

Component 2: The AI Model (The Translator)

This is the **large language model (LLM)** — the brain of the operation.

Examples: * Claude (Anthropic) * GPT-4 (OpenAI) * Qwen (Alibaba) * Any LLM that supports MCP

Role: It interprets your natural language request and figures out which MCP **tools** to call and with what parameters.

Example Interpretation

Your Input: "How many Windows servers do I have?"

AI Model Thinks: "The user wants server counts. I should:

1. Call `inventory_list` to get hosts
2. Filter for hosts with `'os_windows'` group
3. Count them"

Component 3: MCP Server (The Gatekeeper)

The **MCP Server** is the bridge between the AI and your infrastructure.

Role: * Receives structured requests from the AI model * Validates the request using your API token
* Executes the request against your system (AAP, in this case) * Standardizes the response and sends it back to the AI

The MCP server **never trusts the AI directly**. It requires authentication and applies your system’s security policies.

Component 4: Your System (The Executor)

In this course, it’s **Ansible Automation Platform (AAP)**, but it could be:

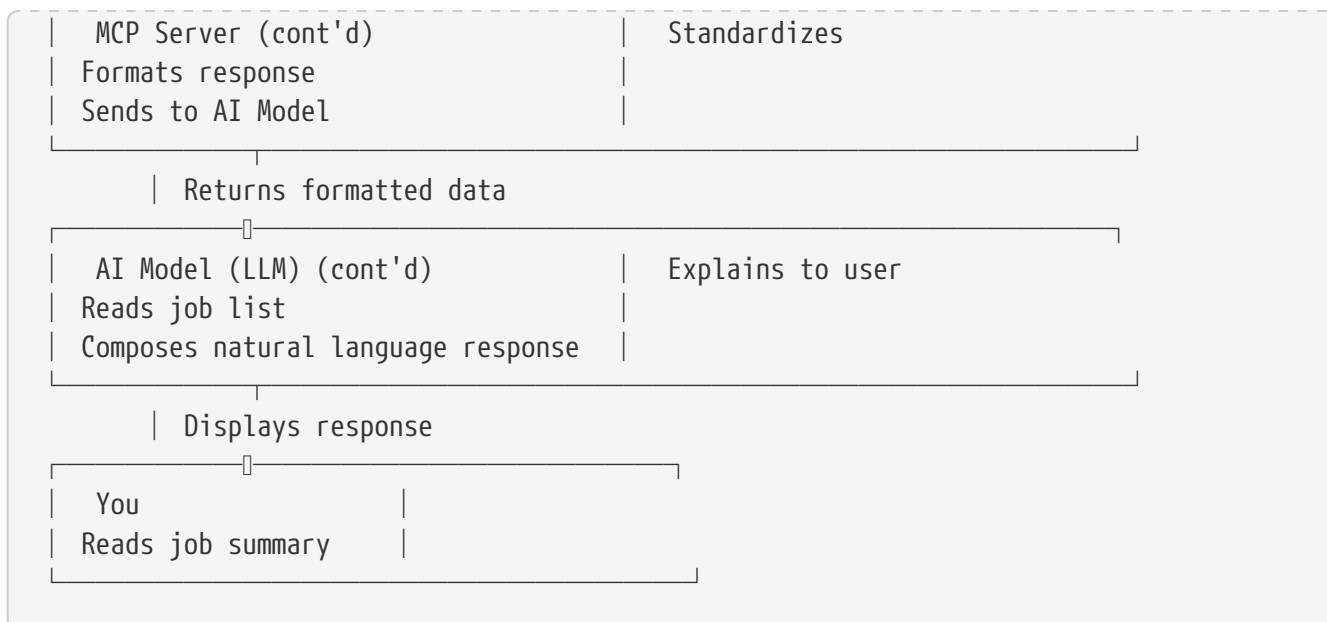
- Your cloud platform (AWS, Azure, GCP)
- Your internal database
- Your monitoring system
- Any system with an API

Role: Executes the actual action or returns the requested data.

How They Connect

Here’s the complete architecture:





What About Security?

The MCP Server is the **security checkpoint**:

1. **Authentication:** Uses your AAP API token to verify requests
2. **Authorization:** Only allows actions your user account has permission to perform
3. **Standardization:** Ensures all requests follow the same format
4. **Validation:** Checks that parameters are valid before sending to AAP

Security Example

AI asks: "Delete all hosts"

MCP Server checks:

- ✓ Is the API token valid? YES
- ✓ Does the user have delete permission? NO
- ✗ DENIED - Permission rejected

Response to AI: "You don't have permission to delete hosts"

Next Steps

You understand the components. Now let's see how **Ansible Automation Platform** uses MCP.

→ **Continue to Section 3: AAP Integration & Toolsets**

AAP Integration & Toolsets

What is the Ansible MCP Server?

The **Ansible MCP Server** is Ansible Automation Platform's implementation of the MCP standard.

It enables connecting AAP with any preferred external AI tool (such as Claude, Cursor, or ChatGPT). The AI tools can access information about your AAP environment and perform tasks using natural language.

Instead of logging into the AAP web UI manually, you can ask an AI: "Launch the 'Deploy Web Servers' job template" and it does it for you.

Why AAP Needs an MCP Server

AAP manages:

- **Thousands of servers and devices**
- **Complex automation workflows and job executions**
- **Sensitive credentials and access control**
- **User teams and role-based permissions**
- **Audit logs and compliance data**

Before MCP, the only way to query this was:

1. Log into the web UI manually
2. Click through menus
3. Run API commands with `curl` or write custom scripts

MCP enables: Ask the AI in natural language, get instant answers.

The 6 AAP MCP Toolsets

The Ansible MCP Server exposes **6 separate toolsets**, each focused on a different area of AAP:

1. Job Management

What it does: Execute and monitor automation jobs.

Tools you can call:

- List job templates
- Start a job run
- Check job status
- View job output/logs
- List recent jobs

Example queries:

```
"Show me failed jobs from the last 24 hours"
"Launch the 'Configure Servers' job template"
```

"What jobs are currently running?"

2. Inventory Management

What it does: Manage hosts, groups, and inventories.

Tools you can call:

- List inventories
- Get hosts in a group
- List inventory groups
- Get host details and variables
- Filter hosts by properties

Example queries:

"Show me all Windows hosts"
"Which hosts are in the 'production' group?"
"List hosts that haven't checked in for 7 days"

3. System Monitoring

What it does: Monitor platform health and performance metrics.

Tools you can call:

- Check AAP health status
- Get execution node metrics
- View job execution statistics
- Monitor capacity and usage
- Get system performance data

Example queries:

"What's the health status of my AAP instance?"
"Show me job execution metrics for the past week"
"What's the capacity utilization?"

4. User Management

What it does: Manage users, teams, and organizations.

Tools you can call:

- List users and teams

- Get team memberships
- View organization details
- Check user roles and permissions
- Manage RBAC assignments

Example queries:

```
"List all users in my organization"  
"Who are the members of the ops team?"  
"What permissions does user 'alice' have?"
```

5. Security & Compliance

What it does: Access security policies, credentials, and audit logs.

Tools you can call:

- View audit logs
- List credentials
- Check security settings
- Review compliance data
- Monitor access and changes

Example queries:

```
"Show me audit logs from the past 24 hours"  
"What credentials are stored in AAP?"  
"List all security-related activity"
```

6. Platform Configuration

What it does: Configure platform settings and resources.

Tools you can call:

- List projects and their status
- View execution environments
- Get platform settings
- Manage integrations
- Configure system properties

Example queries:

```
"List all projects and their SCM status"
"What execution environments are available?"
"Show me platform configuration settings"
```

One Server, Many Endpoints

All 6 toolsets are served by **one MCP server** with **6 separate endpoints**:

Toolset	Endpoint Path
Job Management	/job_management/mcp
Inventory Management	/inventory_management/mcp
System Monitoring	/system_monitoring/mcp
User Management	/user_management/mcp
Security & Compliance	/security_compliance/mcp
Platform Configuration	/platform_configuration/mcp

When you configure an AI tool, you connect it to **all 6 endpoints** so the AI has access to everything.

Role-Based Access Control (RBAC)

The MCP server respects your AAP permissions. If you create an API token for a user with "read-only" permissions, the AI can only read data—it cannot launch jobs or make changes.

RBAC Example

Token created with SCOPE: Read

Allowed:

- ✓ List jobs
- ✓ View job output
- ✓ List inventories
- ✓ View hosts

Denied:

- ✗ Launch a job
- ✗ Create users
- ✗ Delete credentials
- ✗ Modify settings

This means:

- **Admins** can use the AI to do anything they could do manually
- **Read-only users** can only query information
- **Team members** respect team-level permissions

- **Audit logs** track all AI-driven actions

Next Steps

You understand the toolsets. Now let's see the **complete communication workflow**—how a request flows from user to AI to AAP.

→ **Continue to Section 4: Communication Workflow**

Communication Workflow

The Full Journey: From User to AAP

Let's trace exactly what happens when you ask the AI a question. This is the **complete communication workflow**.

Step-by-Step Breakdown

Step 1: The User Initiates a Request

You: "Show me failed jobs from the past 7 days"

This is a natural language request sent to your AI tool (Claude, Cursor, ChatGPT, etc).

What's happening:

- The AI client receives your message
- It sends it to the AI model for interpretation

Step 2: The AI Model Interprets and Plans

```
AI Model (thinking):  
"The user wants:  
1. A list of jobs  
2. Filtered to only FAILED status  
3. From the last 7 days  
  
I need to call the job_list tool with:  
- status = "failed"  
- limit = 7 days  
"
```

What's happening:

- The AI model understands your intent
- It maps your request to available MCP tools

- It prepares the structured request with parameters

Step 3: The MCP Server Receives the Request

Request from AI:

```
{
  "method": "tools/call",
  "params": {
    "name": "job_list",
    "arguments": {
      "status": "failed",
      "limit_days": 7
    }
  }
}
```

What's happening:

- The MCP server receives a structured tool call
- It validates the request format
- It checks that the tool exists

Step 4: The MCP Server Authenticates

MCP Server checks:

- ✓ API token is valid? YES
- ✓ Token belongs to active user? YES
- ✓ User has read permission? YES

Status: AUTHENTICATED

What's happening:

- The MCP server extracts your API token from the request headers
- It verifies the token with AAP
- It loads your user's permissions and RBAC settings

Step 5: The MCP Server Translates and Calls AAP

MCP Server translates to AAP API call:

GET /api/v2/jobs/?status=failed&created__gte=2024-09-08

Headers:

Authorization: Token <your-aap-api-token>

Content-Type: application/json

What's happening:

- The MCP server converts the tool call into an AAP API request
- It uses your API token to authenticate with AAP
- It applies any RBAC restrictions from your user account

Step 6: AAP Executes the Query

AAP Database Query:

```
SELECT jobs WHERE status='failed' AND created > '2024-09-08'
```

RESULT: 3 failed jobs found

What's happening:

- AAP queries its database
- It applies your user's permission restrictions
- It only returns jobs you're allowed to see
- It returns the results as JSON

Step 7: The MCP Server Standardizes the Response

Raw AAP Response (200 JSON objects with 50 fields each):

```
{
  "count": 3,
  "results": [
    {
      "id": 456,
      "name": "Deploy Config",
      "status": "failed",
      "created": "2024-09-10T14:32:00Z",
      "started": "2024-09-10T14:32:05Z",
      "finished": "2024-09-10T14:45:00Z",
      ...50 more fields...
    },
    ...
  ]
}
```

MCP Server Standardizes:

```
{
  "jobs": [
    {
      "id": 456,
      "name": "Deploy Config",
      "status": "failed",
      "duration": "12 minutes",
      "failure_reason": "Host unreachable"
    }
  ]
}
```

```
    },  
    ...  
  ]  
}
```

What's happening:

- The MCP server receives raw AAP data
- It extracts only relevant fields
- It formats data in a way the AI can easily understand
- It simplifies complex structures

Step 8: The AI Model Processes the Data

AI Model receives standardized response
AI Model thinks:
"I have 3 failed jobs. Let me summarize them for the user
in a clear, readable format"

What's happening:

- The AI receives the structured data
- It reads and understands the job details
- It composes a human-readable summary

Step 9: The AI Client Displays the Response

You see in your AI chat:

"You have 3 failed jobs from the past 7 days:

1. Deploy Config (Job #456)
 - Duration: 12 minutes
 - Status: FAILED
 - Reason: Host unreachable
 - Started: Sept 10, 2:32 PM
2. Database Migration (Job #457)
 - Duration: 25 minutes
 - Status: FAILED
 - Reason: Permission denied
 - Started: Sept 11, 10:15 AM
3. Cleanup Tasks (Job #458)
 - Duration: 5 minutes
 - Status: FAILED

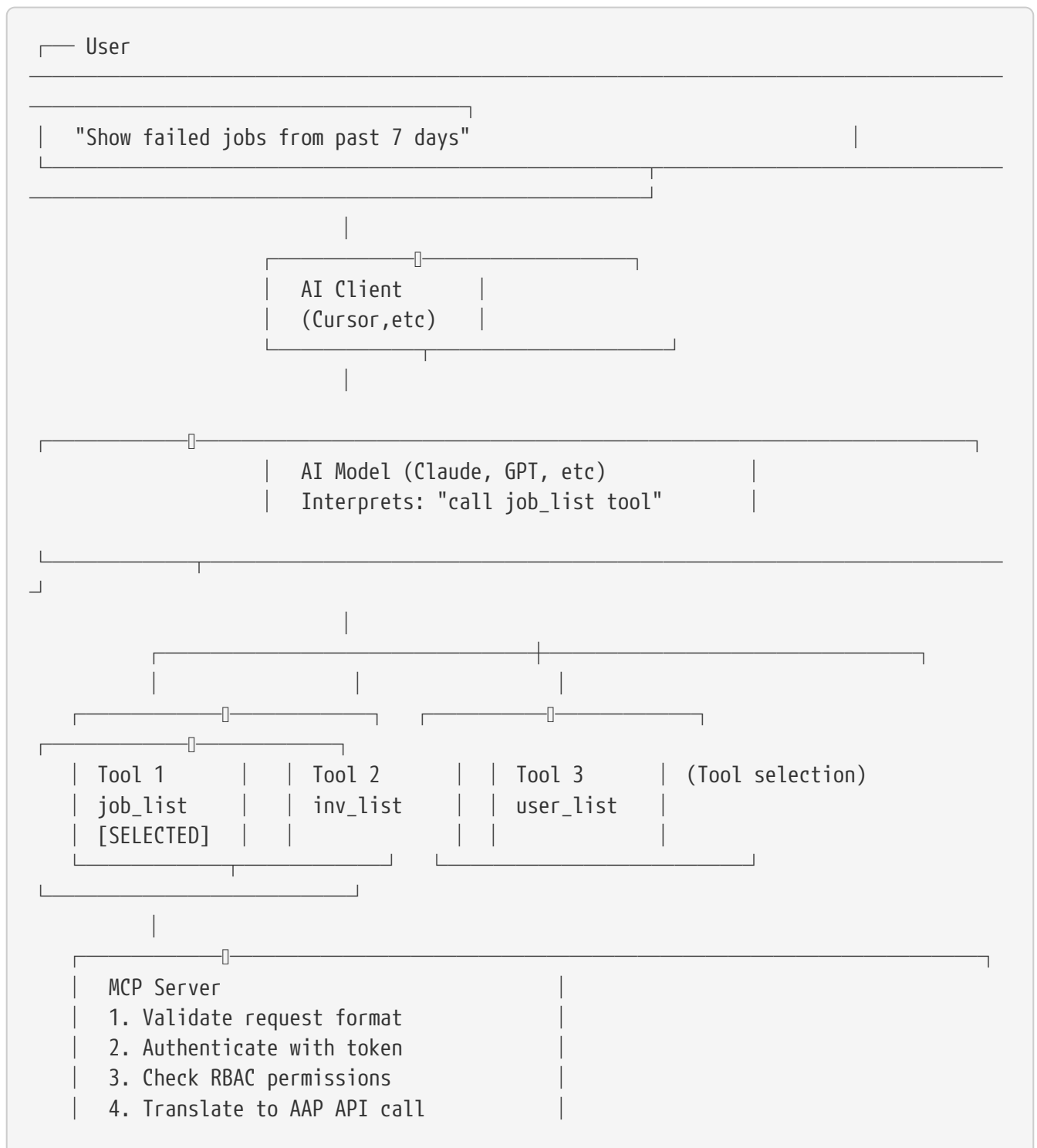
- Reason: Timeout
- Started: Sept 12, 3:45 PM

"

What's happening:

- The AI client displays the formatted response
- You read the summary
- You can now ask follow-up questions

Visualized Workflow





Key Insights

Security Checkpoints

The workflow has **multiple security checkpoints**:

1. ✓ Token validation (is the request from a valid user?)
2. ✓ RBAC check (does the user have permission for this action?)
3. ✓ Data filtering (only show data the user is allowed to see)

Data Flow

Data flows in **two directions**:

1. **Request direction**: Natural language → Structured tool call → API call → Database query
2. **Response direction**: Database result → Formatted response → Natural language summary → Display

No Direct AI-to-AAP Access

Important: The AI model never talks directly to AAP.

The MCP server is always in the middle:

- ✓ The AI cannot bypass authentication
- ✓ The AI cannot exceed user permissions
- ✓ All requests are logged and auditable
- ✓ All data is standardized and validated

Next Steps

You now understand the complete workflow. Let's look at what happens in real deployments.

→ **Continue to Section 5: End-to-End Workflow**

End-to-End Workflow

Getting MCP Running: The Three Steps

Now that you understand **what** MCP is and **how** it works, here's what you actually **do** to make it work.

End to End Workflow

To make use of the Ansible MCP Server, the following steps are required:

1. Deploy the Ansible MCP Server within Ansible Automation Platform
2. Generate an API token for use by the external AI tool
3. Configure the external AI tool to connect to the Ansible MCP Server using the generated API token

Step 1: Deploy the MCP Server

Where Does It Run?

The Ansible MCP Server runs as a containerized service on your infrastructure. You can deploy it on:

- **OpenShift Container Platform (OCP)** — Most common for enterprise
- **RHEL with containers** — Simpler, on a single machine
- **Kubernetes** — Any Kubernetes cluster

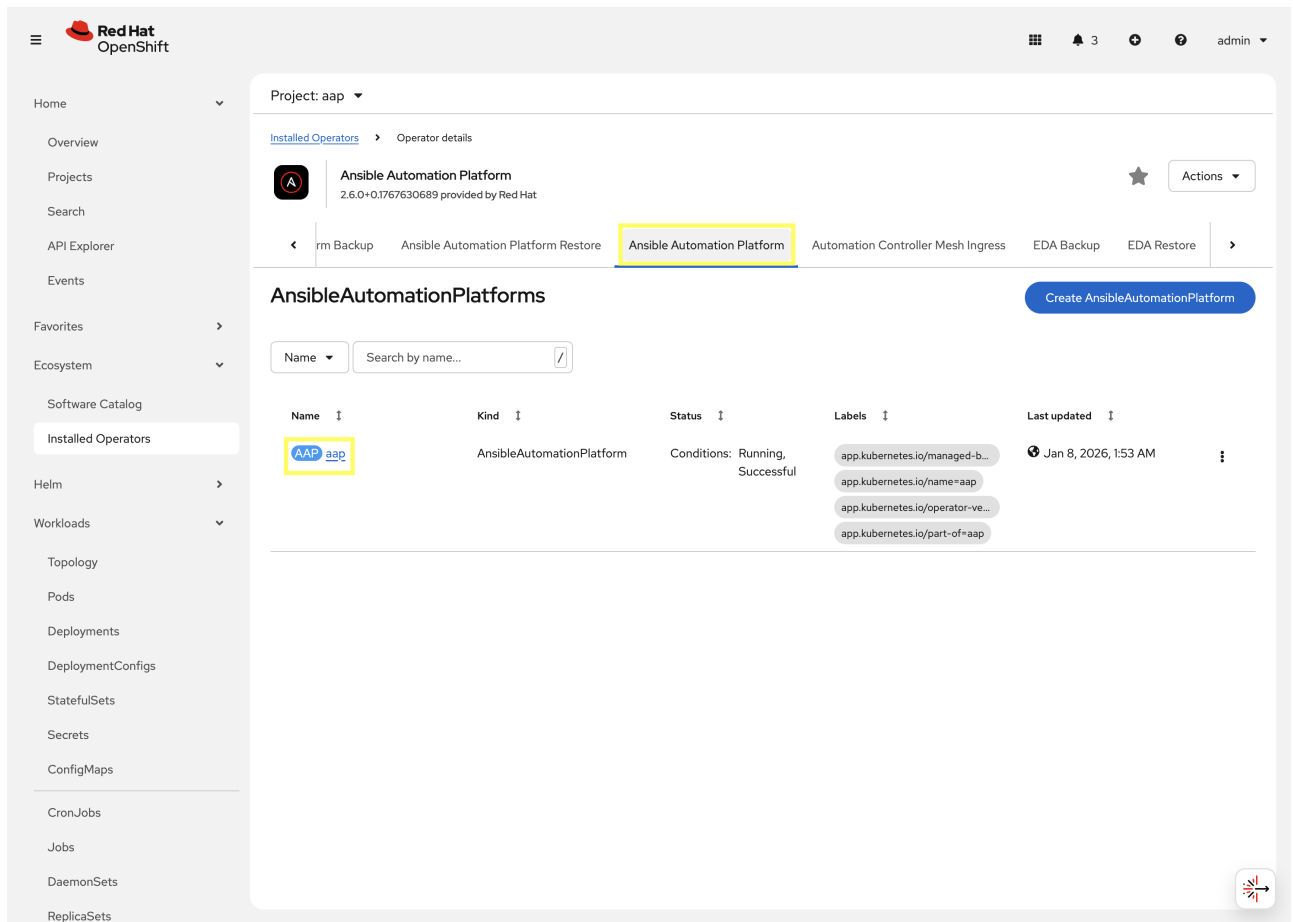
Deploy the Ansible MCP Server to your OpenShift environment by enabling the MCP capability within the **AnsibleAutomationPlatform** custom resource.

1. Launch the [OpenShift Web Console](#)
2. Select the **htpasswd_provider** button and use the credentials provided in the **Environment Details,window=_blank** page to login to the OpenShift console
3. Navigate to **Ecosystem** → **Installed Operators**
4. From the *Project* dropdown, ensure **aap** is selected
5. The **Ansible Automation Platform** operator is listed and installed within this project.

The screenshot shows the OpenShift Web Console interface. On the left is a sidebar with navigation links: Home, Overview, Projects, Search, API Explorer, Events, Favorites, Ecosystem, Software Catalog, Installed Operators, Helm, Workloads, Topology, Pods, Deployments, DeploymentConfigs, StatefulSets, Secrets, ConfigMaps, CronJobs, Jobs, DaemonSets, and ReplicaSets. The main content area is titled 'Installed Operators' and shows a list of operators installed in the 'aap' namespace. The first operator, 'Ansible Automation Platform', is highlighted with a yellow box. Below it are other operators like 'Streams for Apache Kafka', 'Red Hat OpenShift Dev Spaces', 'DevWorkspace Operator', 'Gitea Operator', and 'Red Hat OpenShift GitOps'.

Name	Managed Namespaces	Status	Last updated	Provided APIs
Ansible Automation Platform 2.6.0+0.1767630689 provided by Red Hat	NS aap	✓ Succeeded Up to date	Jan 8, 2026, 2:32 PM	Automation Controller Backup Automation Controller Restore Automation Controller Automation Hub View 20 more...
Streams for Apache Kafka 3.1.0-6 provided by Red Hat	NS aap	✓ Succeeded Up to date	Jan 8, 2026, 2:12 AM	Kafka Kafka Node Pool Kafka Connect Kafka Bridge View 5 more...
Red Hat OpenShift Dev Spaces 3.25.0 provided by Red Hat	NS aap	✓ Succeeded Up to date	Jan 8, 2026, 2:10 AM	Red Hat OpenShift Dev Spaces instance Specification
DevWorkspace Operator 0.38.0 provided by Devfile	NS aap	✓ Succeeded Up to date	Jan 8, 2026, 2:09 AM	DevWorkspace DevWorkspaceTemplate DevWorkspaceOperatorConfig
Gitea Operator 2.1.0 provided by Red Hat Portfolio Technical Marketing and Platforms	NS aap	✓ Succeeded Up to date	Jan 8, 2026, 1:49 AM	Gitea
Red Hat OpenShift GitOps	NS aap	✓ Succeeded Up to date	Jan 8, 2026, 1:48 AM	Argo CD NamespaceManagement

6. Click on the **Ansible Automation Platform** operator which will display all of the resources that are managed by the operator.
7. In the top tab, click on **Ansible Automation Platform** and select **aap**.



8. Click on the **YAML** tab to edit the resource in YAML format.

9. Add the following to the **spec** section by updating the configuration to enable the MCP server:

```
...
spec:
  mcp:
    disabled: false
    allow_write_operations: false
```



Setting **allow_write_operations** to **false** will restrict the MCP server to read-only operations. This is the preferred option, especially in Enterprise environments, when beginning to incorporate AAP with external AI tools. The AI tool may perform unwanted actions if write operations are enabled without fully understanding the implications. For the purpose of this lab, access to write operations will be restricted.

10. Click the **Save** button to apply the changes.

Once the configuration has been applied, the Ansible Automation Platform operator will begin to reconcile the changes. This process can take several minutes to complete.

The OpenShift Operator will then:

1. Create a new Pod for the MCP server

2. Expose it with a network route
3. Handle certificates and networking

How Long Does It Take?

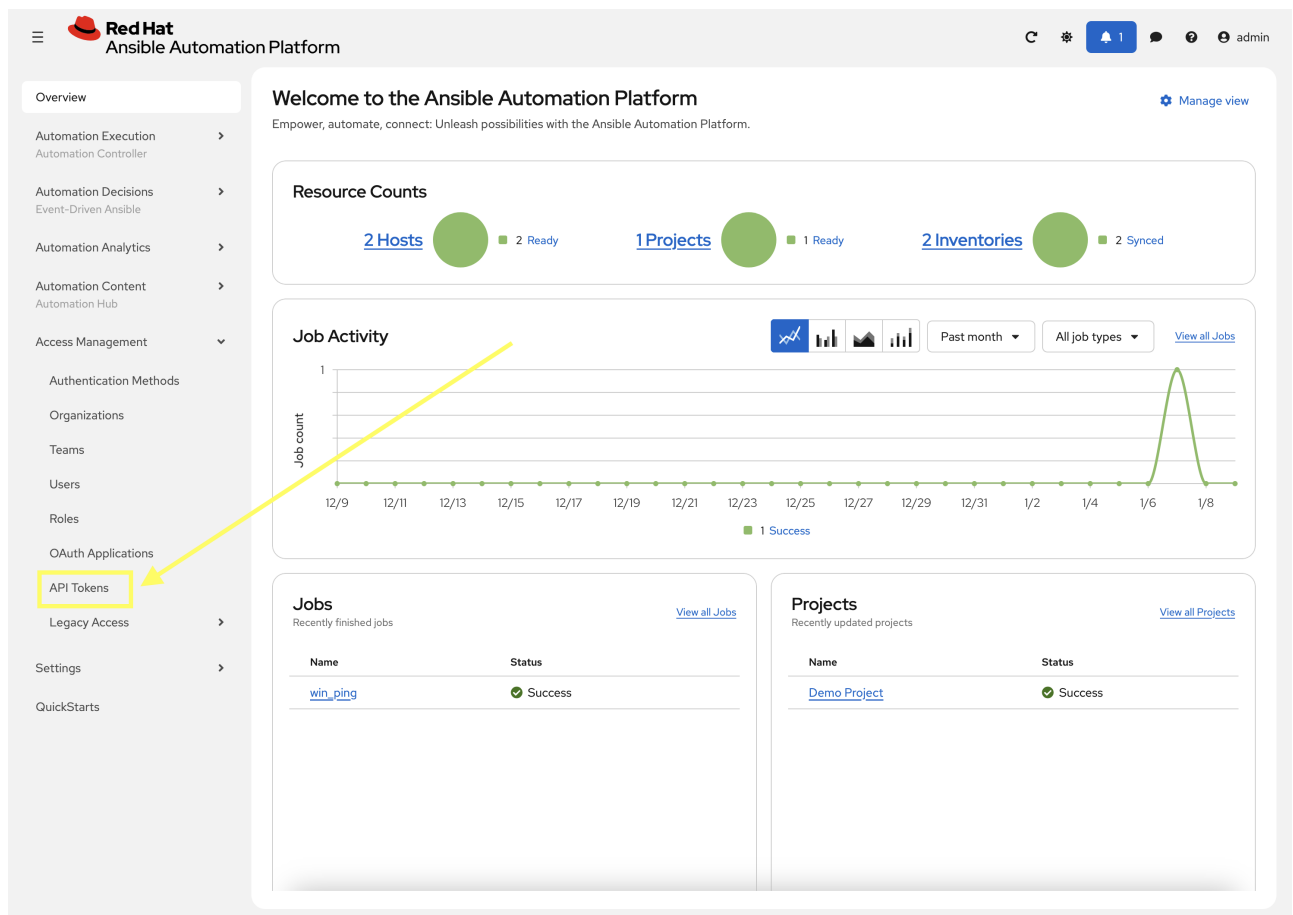
Usually **5-15 minutes** for the operator to reconcile and the MCP server to be ready.

Step 2: Generate an API Token

What's an API Token?

An **API token** is like a password, Create a Personal Access Token (PAT) by performing the following:

1. Launch the [Ansible Automation Platform](#) web interface and login using the credentials from the [Environment Details,window=_blank](#) page.
2. From the navigation menu on the left, expand **Access Management** and click on **API Token**.



3. On the API Token page, click the **Create API Token** button
4. On the Create API Token page, enter the following details:
 - **Description:** `mcp`
 - **OAuth Application:** Leave this field blank which will result in a Personal Access Token (PAT) being created.
 - **Scope:** Select **Read** to restrict the token to read-only operations.

5. Click **Create token** to generate the API token which will be displayed in the resulting dialog



Make sure to store the token securely, as it will not be displayed again.

4: Configuring External AI Tools

Now that the Ansible MCP Server is deployed and an API token has been generated, the final step is to configure external AI tools that can be used to connect to the Ansible MCP Server using the generated API token. While there are multiple external AI tools that can be used to demonstrate the integration with the Ansible MCP Server, for the purpose of this lab, the following solutions will be demonstrated:

1. [OpenCode](#) - Terminal based AI agent that can be executed from within the OpenShift Dev Spaces workspace.
2. [LibreChat](#) - Web based AI chat tool that can be accessed via a web browser.

4.1: Using OpenCode with the Ansible MCP Server

[OpenCode](#) is an Open Source, terminal based AI agent. It is already installed in your Dev Spaces workspace and requires that a few steps be completed prior to its use.

4.1.1: Customizing a Dev Spaces Workspace

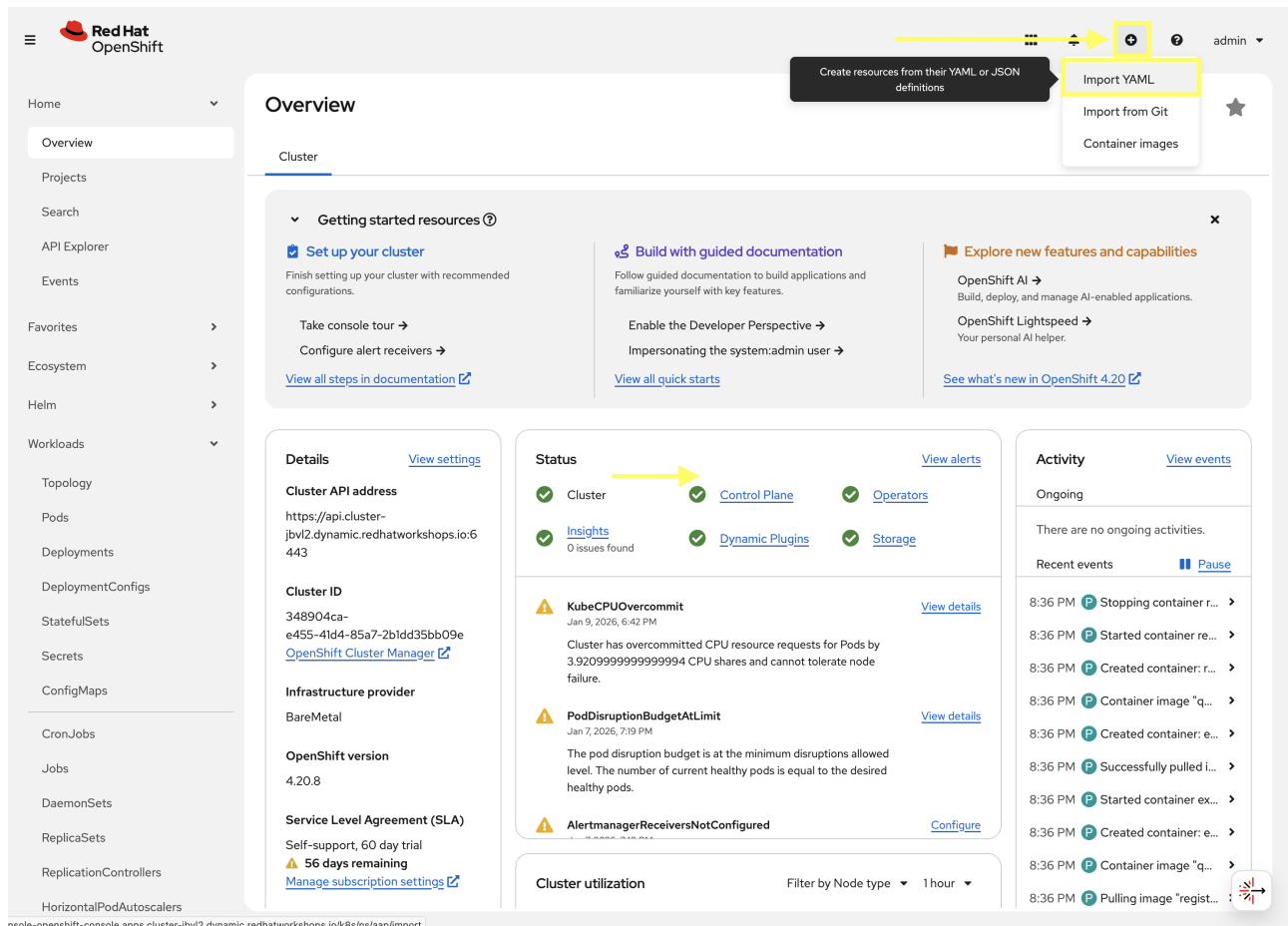
OpenShift Dev Spaces includes several different methods for configuring and customizing a workspace. For this section, we will create a `ConfigMap` and `Secret` resource within the `admin-devspaces` project containing your workspace to externalize and inject environment variables into

your workspace. These values will be referenced by the OpenCode configuration file that you will then define to drive the functionality of the tool.

Labels and annotations are applied to the **ConfigMap** and **Secret** resources so that they become eligible for injection into the Dev Spaces workspace. By setting the **controller.devfile.io/mount-to-devworkspace** label to **true**, the Dev Spaces controller will automatically mount the resources into the workspace. The contents of the **ConfigMap** or **Secret** can either be mounted as files or environment variables. This is dictated by the **controller.devfile.io/mount-as** annotation.

First, create a **ConfigMap** resource to contain the non-sensitive configuration values by performing the following in the OpenShift web console. Instead of using the OpenShift web console to interactively create the ConfigMap, you will import the raw YAML definition using the **Import YAML** feature.

1. Launch the **OpenShift Web Console**
2. Select the **htpasswd_provider** button and use the credentials provided in the **Environment Details,window=_blank** page to login to the OpenShift console
3. On the top navigation bar, click on the + icon and select **Import YAML**



4. Enter the following into the **Import YAML** dialog:

```
kind: ConfigMap
apiVersion: v1
metadata:
  name: superlab-devspaces-config
```

```

namespace: admin-devspaces
labels:
  controller.devfile.io/mount-to-devworkspace: 'true'
  controller.devfile.io/watch-configmap: 'true'
annotations:
  controller.devfile.io/mount-as: env
data:
  AAP_MCP_URL: https://aap-mcp-aap.{openshift_cluster_ingress_domain}/mcp
  LITELLM_MODEL_NAME: {litellm_available_models_list}
  LITELLM_URL: {litellm_api_base_url}
  OPENCODE_CONFIG: /projects/ansible-devspaces/opencode.json

```



Notice the value of the `controller.devfile.io/mount-as` annotation in the snippet above is set to `env`, which means the contents of the `ConfigMap` will be mounted as environment variables in the Dev Spaces workspace.

5. Click the **Create** button to create the `ConfigMap` resource.

Now, create a `Secret` resource to contain the sensitive configuration values by repeating the same steps as above by using the *Import YAML* feature.

1. On the top navigation bar, click on the + icon and select **Import YAML**
2. Enter the following into the *Import YAML* dialog. Make sure to replace `PLACEHOLDER_REPLACE_ME` with the AAP API Token you generated in the prior section:

```

kind: Secret
apiVersion: v1
metadata:
  name: superlab-devspaces-secret
  namespace: admin-devspaces
  labels:
    controller.devfile.io/mount-to-devworkspace: 'true'
    controller.devfile.io/watch-secret: 'true'
  annotations:
    controller.devfile.io/mount-as: env
stringData:
  AAP_MCP_API_KEY: PLACEHOLDER_REPLACE_ME
  LITELLM_API_KEY: {litellm_virtual_key}
type: Opaque

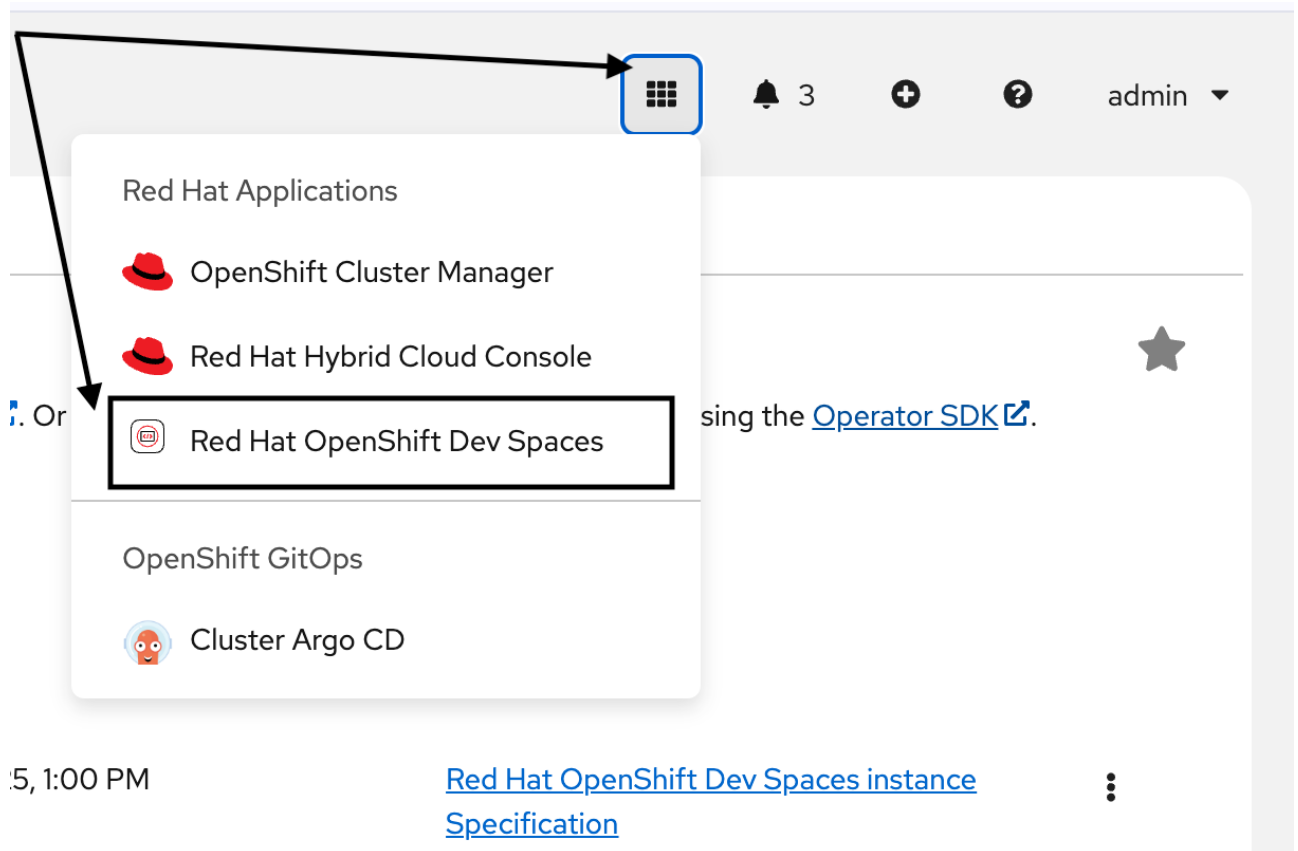
```

3. Click the **Create** button to create the `Secret` resource.

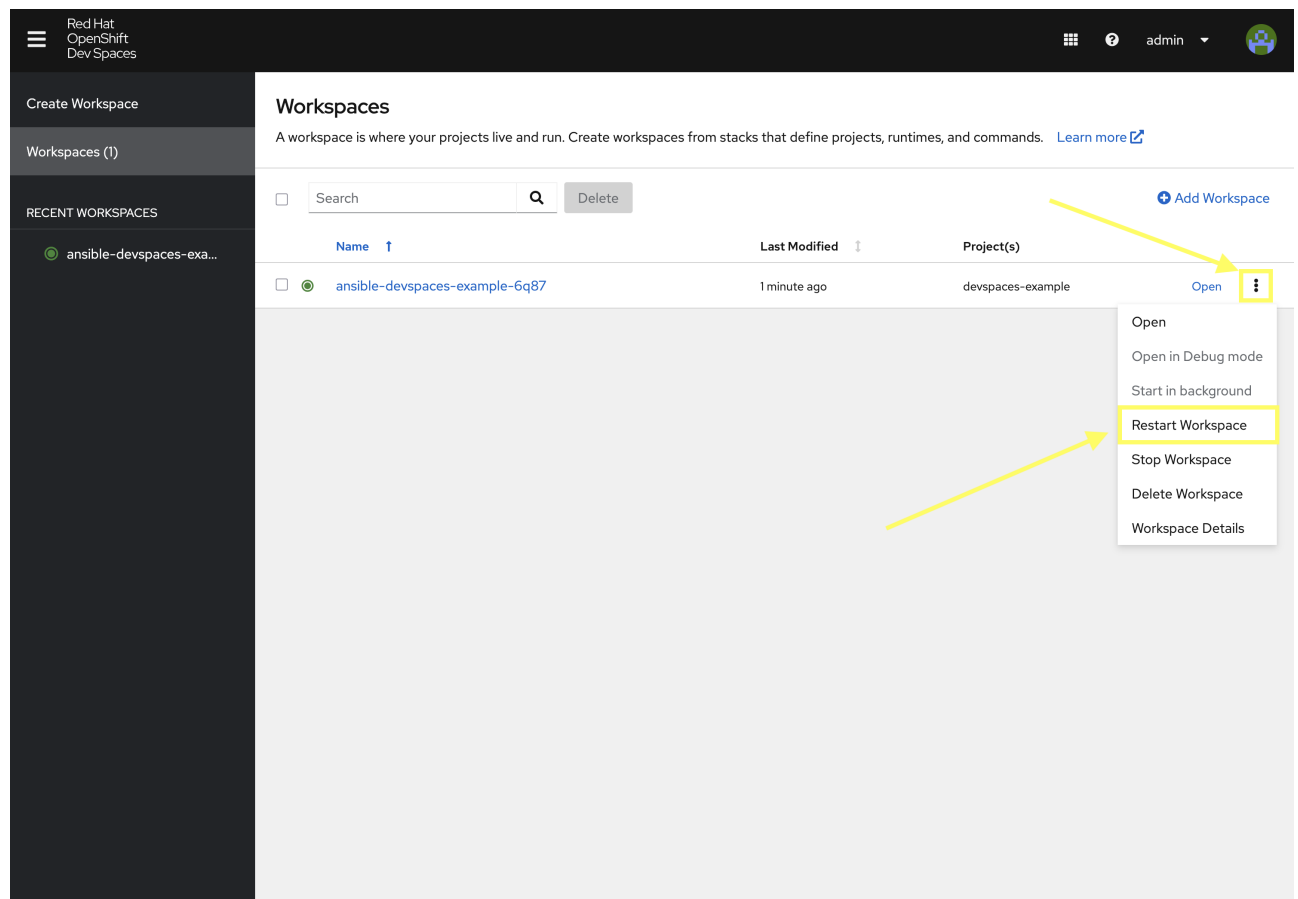
With the environment variables defined in the `ConfigMap` and `Secret` now defined within OpenShift, the next step is to restart your Dev Spaces workspace so that it can mount the values contained within these resources. To restart your Dev Spaces workspace, perform the following steps in the OpenShift web console:

1. Use the 9-block icon in the upper right to launch Red Hat OpenShift Dev Spaces directly from

the OpenShift console header.



2. Within the Dev Spaces console, on the *Workspaces* page, click the kebab menu (three vertical dots) next to your `ansible-dev-spaces-example-xxxx` workspace and select **Restart Workspace**.



3. Click the **Open** link next to your workspace to relaunch the workspace

When the workspace has started, you can verify that the environment variables are correctly mounted by executing the following command in the terminal which will check for the presence of the environment variables as defined in the **ConfigMap** and **Secret** resources:

```
env | grep -E "(AAP_MCP|LITELLM)"
```

The output should display values that correspond to the environment variables defined in the **ConfigMap** and **Secret** resources previously.

4.1.2: Configuring OpenCode

With the necessary environment variables now mounted into your Dev Spaces workspace, the next step is to introduce the external AI tool that will interact with Ansible Automation Platform via the Ansible MCP Server. For this section, we will utilize **OpenCode**, an open-source AI development platform that enables developers to build, deploy, and manage AI applications. OpenCode provides a unified interface for interacting with various AI models and services, making it easier to integrate AI capabilities into applications.

You may have noticed earlier that one of the environment variables defined in the **ConfigMap** resource was the **OPENCODE_CONFIG** property, which specifies the location of the OpenCode configuration file (Hint: you can run the command `env | grep OPENCODE_CONFIG` to see the value). This file does not exist yet as we will need to create it as part of this exercise. An **OpenCode configuration file** contains details related to the inference service as well as any MCP servers that should be utilized.

Create a new file named **opencode.json** at the within the **/projects/ansible-devspaces** directory containing the following:

/projects/ansible-devspaces/opencode.json

```
{
  "$schema": "https://opencode.ai/config.json",
  "autoupdate": false,
  "enabled_providers": [
    "LiteLLM"
  ],
  "mcp": {
    "aap": {
      "type": "remote",
      "enabled": true,
      "oauth": false,
      "url": "{env:AAP_MCP_URL}",
      "headers": {
        "Authorization": "Bearer {env:AAP_MCP_API_KEY}"
      }
    }
  },
}
```

```

"provider": {
  "LiteLLM": {
    "npm": "@ai-sdk/openai-compatible",
    "name": "LiteLLM",
    "options": {
      "baseUrl": "{env:LITELLM_URL}",
      "apiKey": "{env:LITELLM_API_KEY}"
    },
    "models": {
      "qwen3-14b": {
        "name": "{env:LITELLM_MODEL_NAME}",
        "limit": {
          "context": 35000,
          "output": 1000
        }
      }
    }
  }
}

```

In this configuration file, we are specifying details related to our inference service along with the AAP MCP Server. Take note of the variables beginning with `{env:…}` which will replace the values with the content injected from the *Secret* or *ConfigMap* defined previously.

4.1.3: Using OpenCode to Interact with the Ansible MCP Server

With all of the configuration steps complete, let's utilize OpenCode to interact with the Ansible MCP Server.

To launch OpenCode, execute the following command:

```
opencode
```

Like many other AI coding tools, OpenCode is a Text User Interface (TUI) based tool which allows you to interact directly with the tool. By launching OpenCode, you will be placed at a prompt for which you can enter commands or execute queries:



As you can see, OpenCode is using the LiteLLM inference service along with the `{litellm_available_models_list}` Model that was defined in the *ConfigMap*.

Use the `/status` command to view the status of the tool including any integration with external components, such as the AAP MCP server.

```
/status
```

If your MCP server is running and the API token that was stored in the *Secret* is being loaded properly, the status being displayed should look similar to the following:



The `aap` MCP server displays *Connected* indicating that it is able to integrate with Ansible Automation Platform as needed. If the status being displayed indicates an error, investigate any message/error code that may be present. In addition, you can view the logs from the AAP MCP Server pod that is running in the `aap` Project. The pod will have a name of `<instance_name>-mcp-xxxx` which may provide additional insights related to the failed integration.

Hit the `ESC` key to exit the status dialog to return to the main prompt.

Now, let's start a chat session using our natural language that retrieves details that originate from Ansible Automation Platform that is facilitated through the integration with the AAP MCP Server.

Let's ask for information related to the *Projects* that have been defined:

```
Display the names of aap projects
```

In a few moments, you will see OpenCode making queries to the *Tools* exposed by the AAP MCP Server and then providing a natural language response based on the information retrieved.

A successful invocation of the `project_list` MCP tool will display similar to the following:



Either the list of AAP project that are returned as a result of the `projects_list` tool invocation will be displayed or you can continue the conversation by asking additional questions to display more information related to the desired query.

If an error or returned or no data is displayed, you can review the [troubleshooting section](#) below for additional guidance.



Due to the non-deterministic nature of LLM's, the exact response provided by OpenCode may vary between invocations and may not match exactly what is shown in the screenshot or steps above.

4.1.4: Troubleshooting OpenCode MCP Integration

If you encounter issues when attempting to interact with the Ansible MCP Server via OpenCode, consider the following troubleshooting steps:

1. Response similar to `Your current request is incomplete. Please provide more details or specify the task you need help with`
 - This response typically indicates that the LLM model was unable to interpret the request or map it to an available MCP tool. Feel free to retry the request again.



Hitting the up arrow key will place the last request into the chat window for easy access for subsequent invocations.

2. No response is provided
 - Retry the request again
3. Abort the current request
 - If the request appears to be taking too long to process, or you wish to abort the current request, hit the `ESC` key to interrupt the current operation.
4. Clear the existing context/start a new chat session
 - Enter `/new` into the chat prompt to start a new session

4.2: Using LibreChat with the Ansible MCP Server

LibreChat is a Open Source AI chat tool that can be integrated with various hosted and self-hosted LLM's. LibreChat can be accessed via a web browser and provides a chat-based interface for interacting with AI models. This section will illustrate how to configure LibreChat to connect to the Ansible MCP Server.

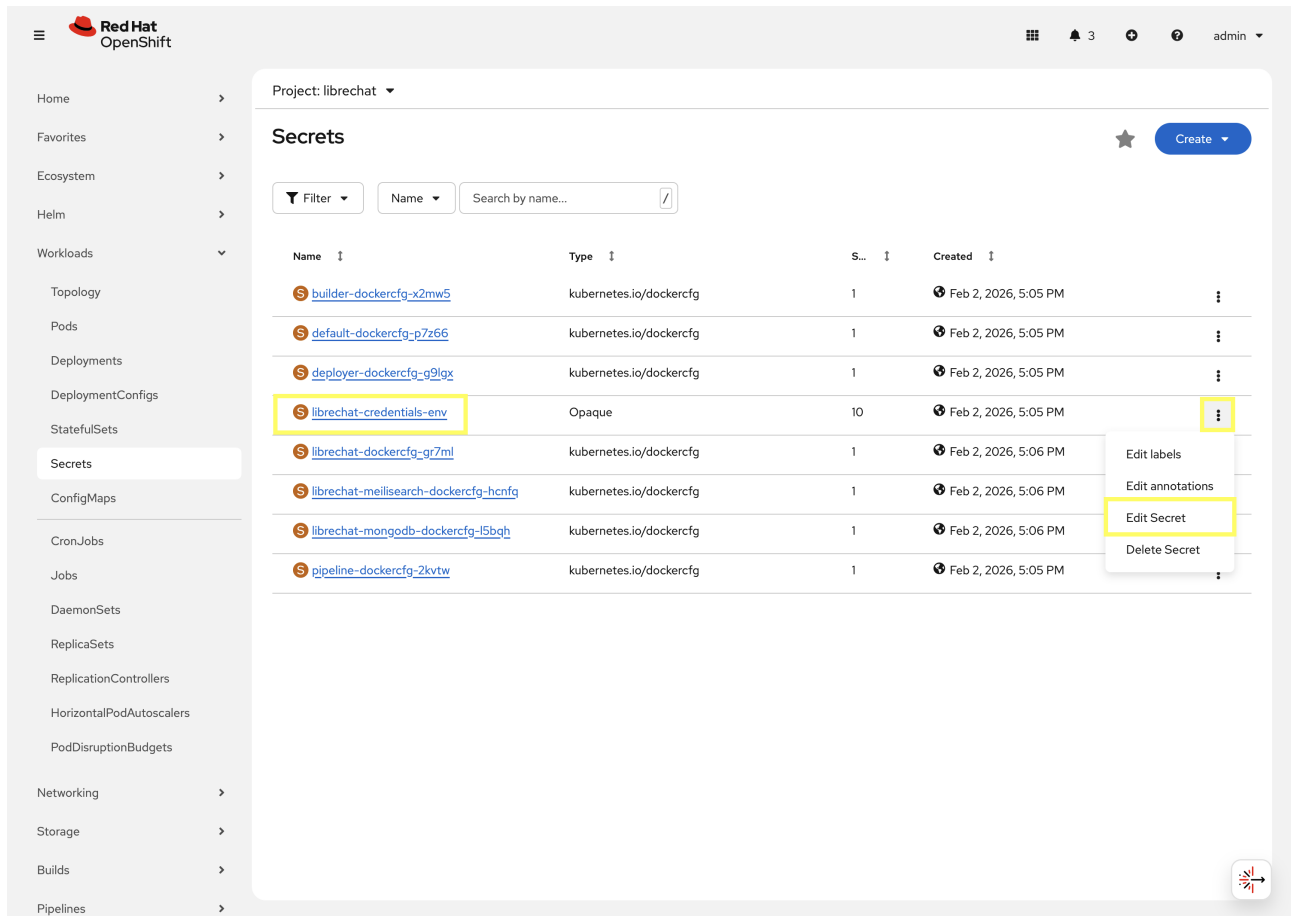
4.2.1: Configuring LibreChat

LibreChat has been deployed within the OpenShift environment as part of the lab setup. While most of the configurations have been pre-configured for you, there is one final step that you need to complete to enable the integration with the Ansible MCP Server: Specifying the API token that was generated in the prior section.

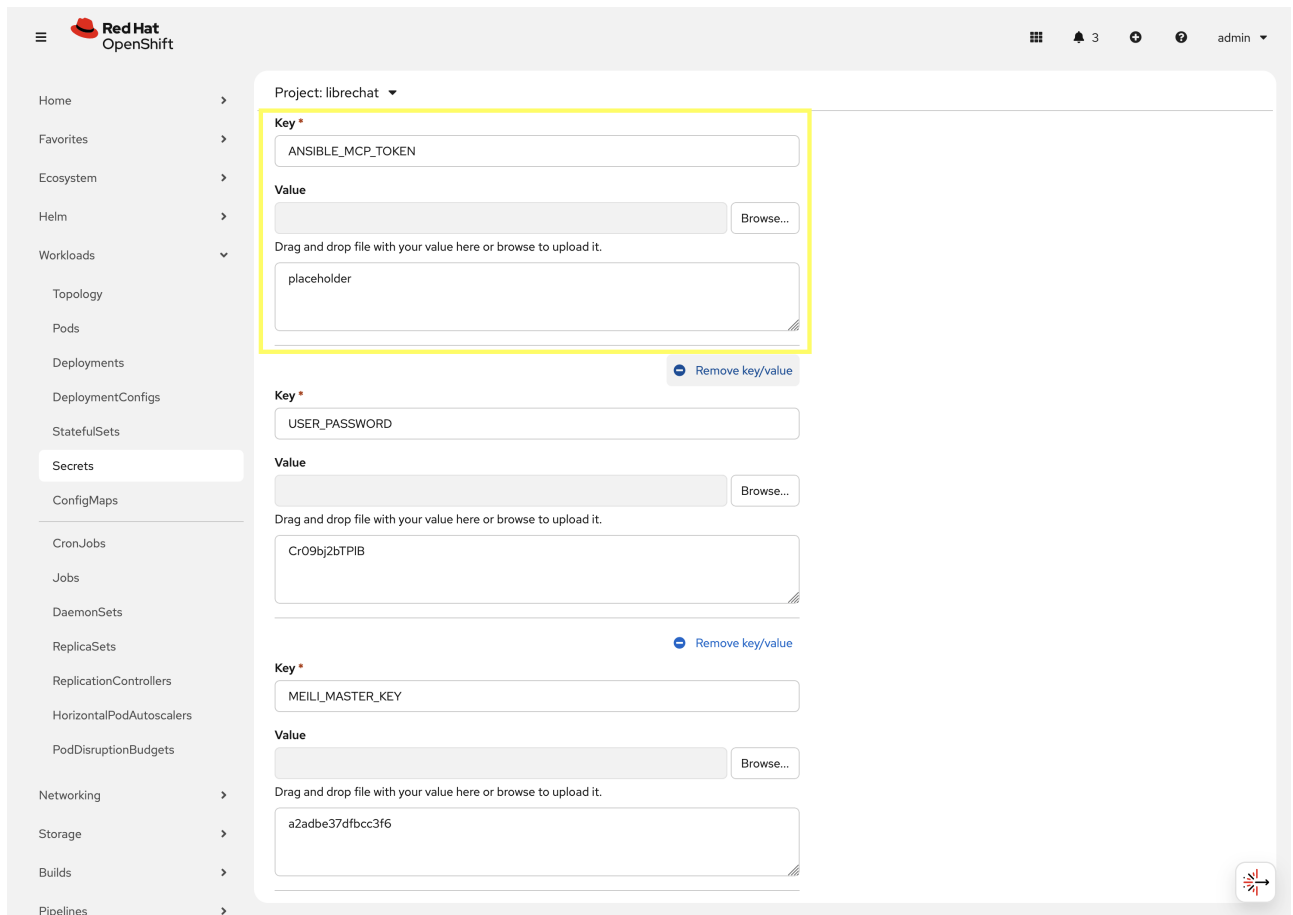
All of the resources associated with LibreChat have been deployed in the `librechat` Project. To configure the Ansible MCP API token, perform the following steps in the OpenShift web console to update the value stored in the `librechat-credentials-env` Secret resource in the `librechat` Project:

1. Launch the [OpenShift Web Console](#)
2. Select the `htpasswd_provider` button and use the credentials provided in the `Environment Details,window=_blank` page to login to the OpenShift console

3. Navigate to **Workloads** on the left hand navigation → **Secrets**. From the project dropdown at the top, select **librechat**.
4. Locate the **librechat-credentials-env** Secret resource, click the kebab menu (three vertical dots) on the right hand side of the entry and select **Edit Secret**.



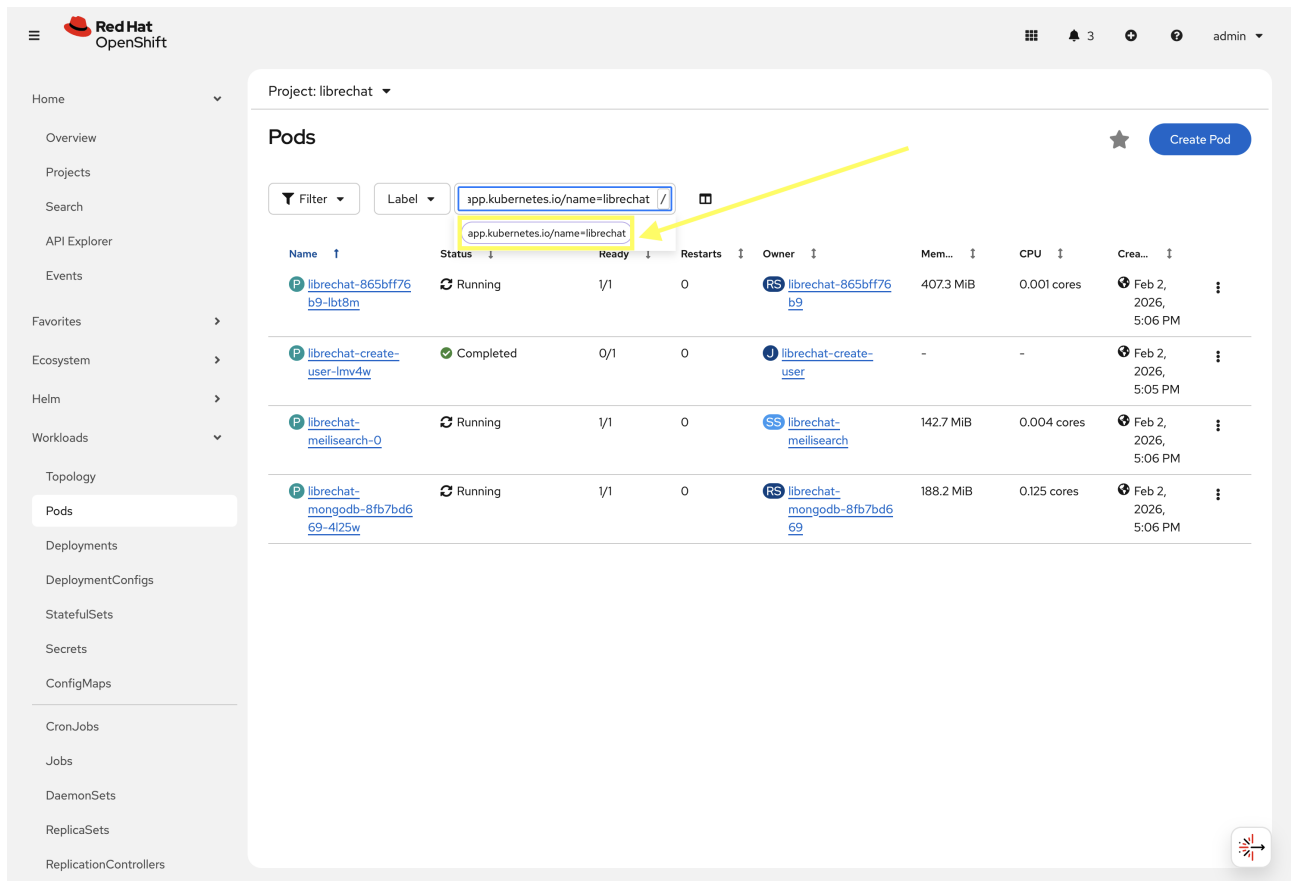
5. Locate the **ANSIBLE_MCP_TOKEN** entry. By default, the value is set as **placeholder**. Update the value of this entry by replacing **placeholder** with the API token that was generated in the prior section.



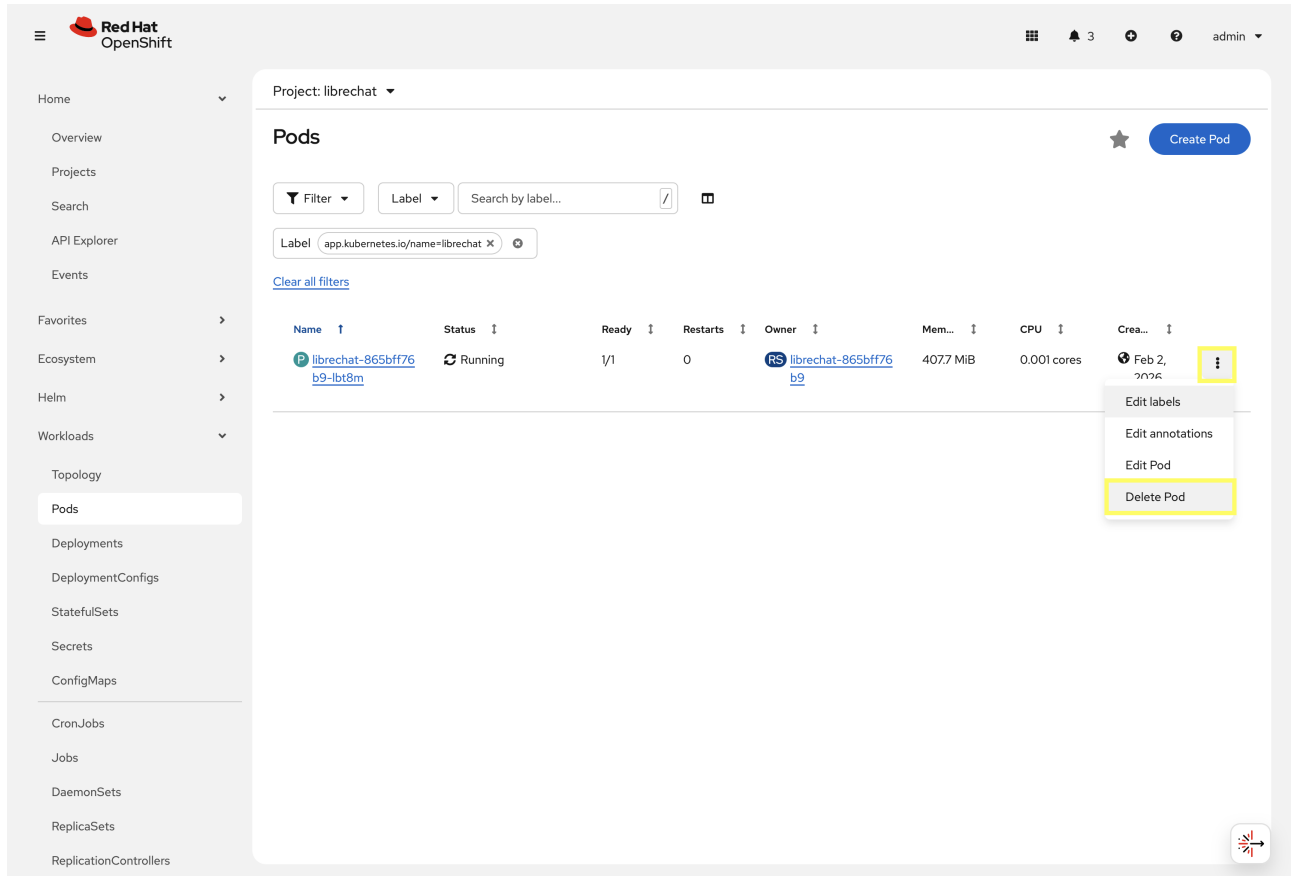
6. Scroll to the bottom of the page and click the **Save** button to apply the changes.

Once the *Secret* containing the API token has been updated, the next step is to restart the LibreChat pod so that it can load the updated value. To restart the LibreChat pod, perform the following steps:

1. Navigate to **Workloads** on the left hand navigation → **Pods**. From the project dropdown at the top, ensure **librechat** is selected.
2. Locate the filter section at the top of the page and select the dropdown that has the value of **Name** present and select **Label**. Enter **app.kubernetes.io/name=librechat** in the text box and select the button provided below the textbox to apply the filter.



- Locate the single pod that is displayed and click the kebab menu (three vertical dots) on the right hand side of the entry and select **Delete Pod**.



- Wait until the pod has been recreated, is *Running*, and displays *1/1* in the *Ready* column.

5. Launch the LibreChat web interface by navigating to: [LibreChat Web Interface](#). You will be presented with the LibreChat login page.



Welcome back

Email address

admin@example.com

Password

••••••••

Continue

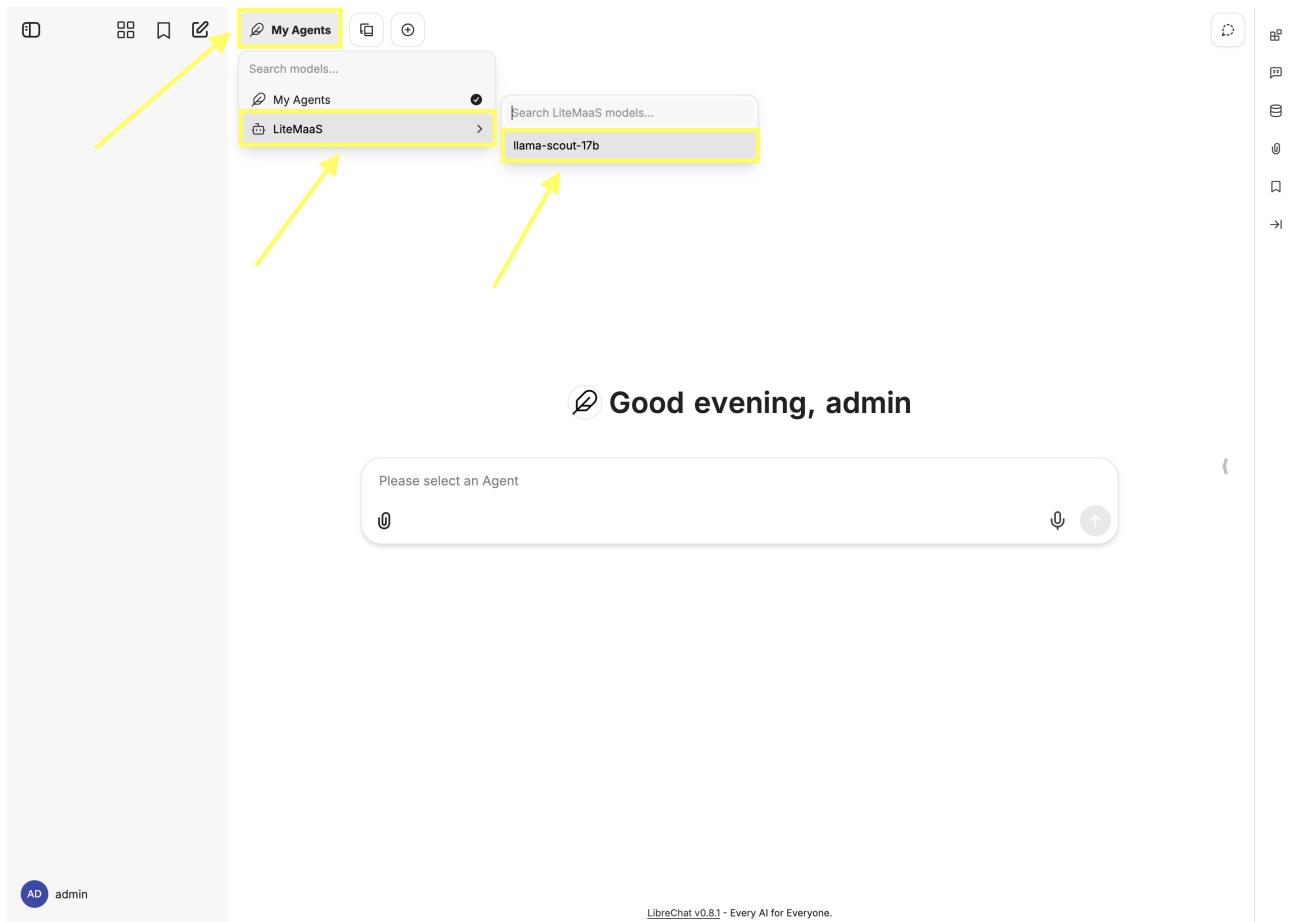
Don't have an account? [Sign up](#)



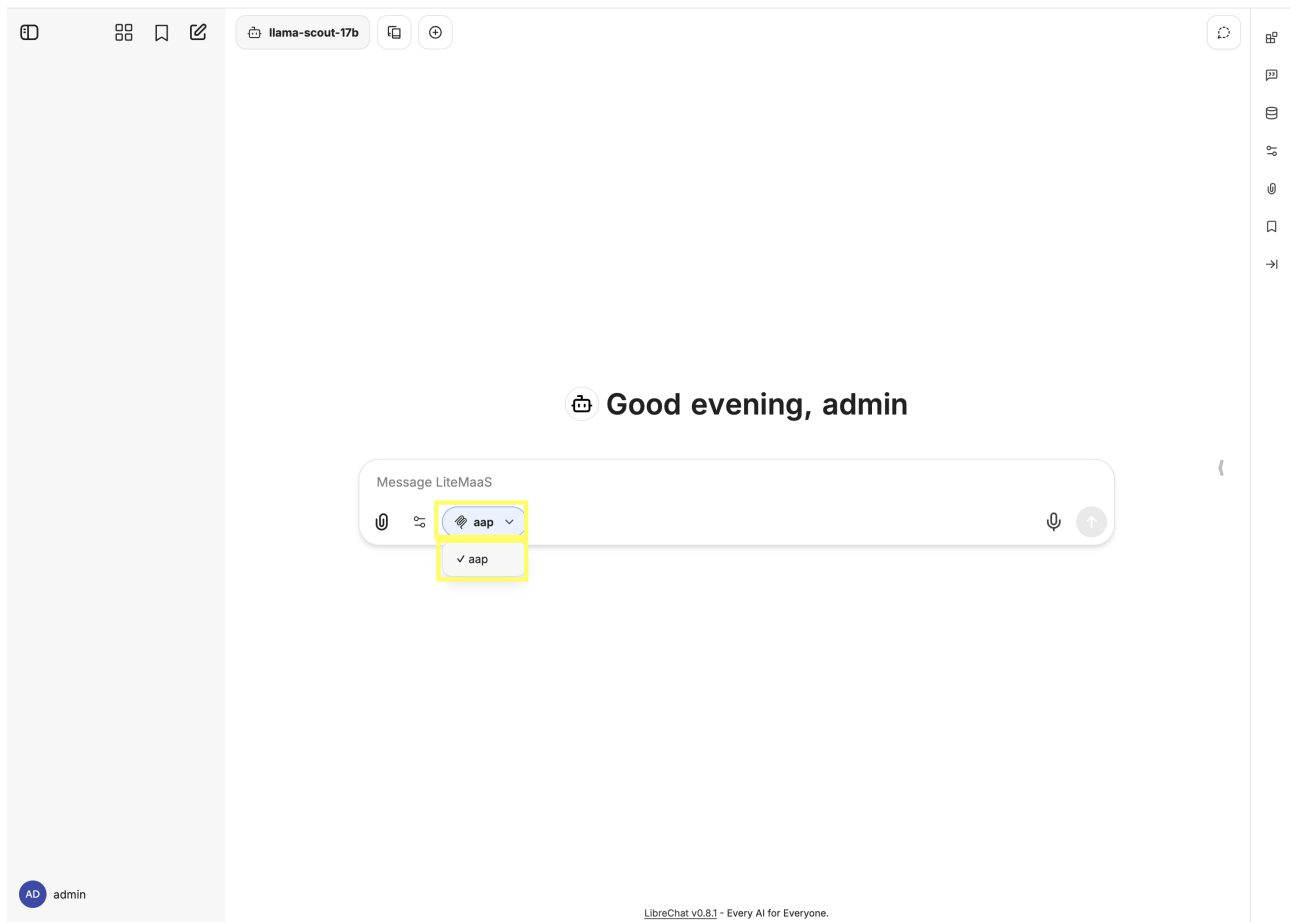
6. Enter the username and password using the credentials provided from the [Environment Details,window=_blank](#) page and click the **Continue** button.

Once authenticated, you will be presented with the LibreChat main interface.

7. Enable the {litellm_available_models_list} model by clicking on the **My Agents** button at the top of the page, hover over **LiteMaaS**, and then select {litellm_available_models_list}.



8. Enable the connection to the Ansible MCP Server by selecting the *MCP Servers* dropdown within the chat window and selecting **aap**. A successful connection will display a checkmark next to the MCP server name.

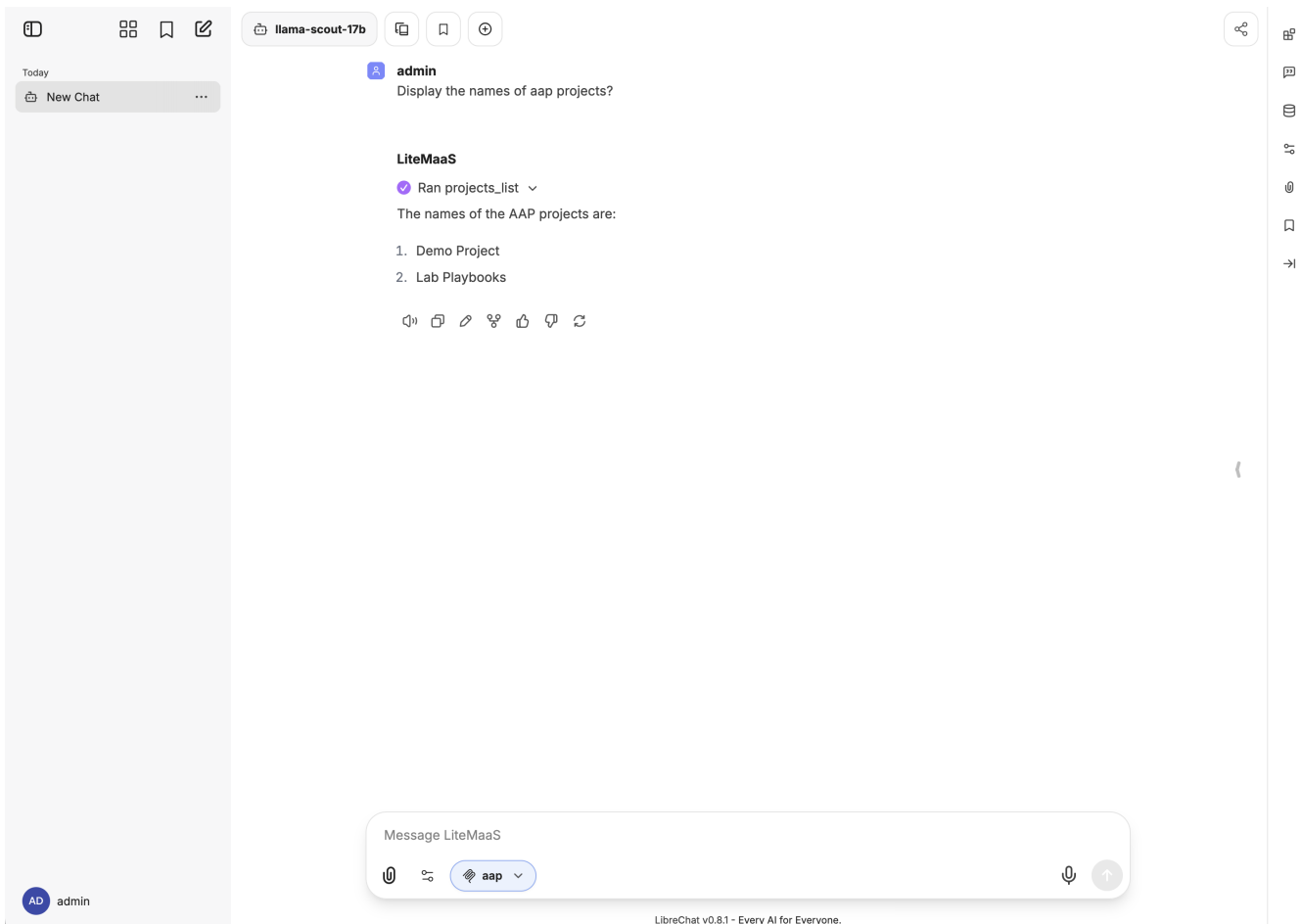


Now, let's start a chat session using our natural language that retrieves details that originate from Ansible Automation Platform that is facilitated through the integration with the AAP MCP Server.

9. Ask for information related to the *Projects* that have been defined:

Display the names of aap projects

In a few moments, you will see LibreChat making queries to the *Tools* exposed by the AAP MCP Server and then providing a natural language response based on the information retrieved. A successful invocation of the MCP tool will display similar to the following:



Notice how the `projects_list` tool was invoked to retrieve the list of projects from Ansible Automation Platform. The response provided by LibreChat includes the names of the projects that were retrieved. You can expand the result of the `projects_list` tool to see the raw data that was returned from Ansible Automation Platform if desired.



Due to the non-deterministic nature of LLM's, the exact response provided by LibreChat may vary between invocations and may not match exactly what is shown in the screenshot or steps above.

Feel free to continue the chat session or start a new session to continuing to converse with the LLM, obtaining realtime insights from Ansible Automation Platform though the use of the Ansible MCP Server.

section6.adoc

Key Takeaways

- **MCP provides a standard way** to connect AI models with external systems.
- **Ansible MCP Server connects AI with AAP**, exposing AAP capabilities through MCP toolsets.
- **The MCP Server acts as the gatekeeper**, handling requests between the AI client and AAP.
- **Authentication and RBAC are enforced**, ensuring users can only access and perform actions permitted by their AAP permissions.
- **AAP data is standardized**, providing the AI with consistent and easier-to-process responses.

- **Natural-language interaction simplifies automation**, reducing the need for manual UI navigation or direct API calls.

Chapter 2: Hands-On Labs

Lab 1: Deploy MCP on OCP

Lab 2: Configure Your AI Tool

Lab 3: Run Queries & Troubleshoot

Appendix & Resources

MCP Cheat Sheet

Knowledge Check & Quiz

Troubleshooting Guide